

22. Spring Boot Testing

22.1. Giới thiệu về Testing trong Spring Boot

Testing là một phần không thể thiếu trong quá trình phát triển phần mềm, đảm bảo chất lượng, độ tin cậy và tính đúng đắn của ứng dụng. Spring Boot cung cấp một bộ công cụ mạnh mẽ và tích hợp sẵn để hỗ trợ việc kiểm thử các ứng dụng Spring, từ unit tests đến integration tests và slice tests.

Spring Boot Testing giúp đơn giản hóa việc thiết lập môi trường kiểm thử, tự động cấu hình các thành phần cần thiết và cung cấp các tiện ích để viết các bài kiểm thử hiệu quả.

22.2. Các loại Testing trong Spring Boot

22.2.1. Unit Testing (Kiểm thử đơn vị)

Unit testing tập trung vào việc kiểm thử các đơn vị mã nguồn nhỏ nhất và độc lập (ví dụ: một lớp, một phương thức) mà không cần khởi động toàn bộ Spring ApplicationContext. Mục tiêu là kiểm tra logic nghiệp vụ của từng thành phần một cách riêng biệt.

- **Công cụ:** JUnit 5 (mặc định), Mockito.
- **Đặc điểm:** Nhanh, cô lập, không phụ thuộc vào Spring Context.

Ví dụ: Kiểm thử một Service không phụ thuộc vào Spring Data JPA hay Controller.

Java

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.util.Arrays;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;
```

```
// Giả sử có một ProductRepository và ProductService
class Product {
    private Long id;
    private String name;
    // Constructors, Getters, Setters
    public Product(Long id, String name) { this.id = id; this.name = name; }
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

interface ProductRepository {
    List<Product> findAll();
    Product save(Product product);
}

class ProductService {
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Product createProduct(Product product) {
        return productRepository.save(product);
    }
}

public class ProductServiceUnitTest {

    @Mock // Tạo một mock object cho ProductRepository
    ProductRepository productRepository;

    @InjectMocks // Inject mock object vào ProductService
    ProductService productService;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this); // Khởi tạo các mock
    }

    @Test
```

```

void testGetAllProducts() {
    // Định nghĩa hành vi của mock
    when(productRepository.findAll()).thenReturn(Arrays.asList(
        new Product(1L, "Laptop"),
        new Product(2L, "Mouse")
    ));

    List<Product> products = productService.getAllProducts();
    assertEquals(2, products.size());
    assertEquals("Laptop", products.get(0).getName());
}

@Test
void testCreateProduct() {
    Product newProduct = new Product(null, "Keyboard");
    Product savedProduct = new Product(3L, "Keyboard");

    when(productRepository.save(newProduct)).thenReturn(savedProduct);

    Product result = productService.createProduct(newProduct);
    assertEquals(3L, result.getId());
    assertEquals("Keyboard", result.getName());
}
}

```

22.2.2. Integration Testing (Kiểm thử tích hợp)

Integration testing kiểm thử sự tương tác giữa các thành phần khác nhau của ứng dụng (ví dụ: Controller với Service, Service với Repository, ứng dụng với database). Các bài kiểm thử này thường yêu cầu khởi động một phần hoặc toàn bộ Spring ApplicationContext.

- **Công cụ:** `@SpringBootTest`, `@Autowired`, `TestRestTemplate`, `WebTestClient`.
- **Đặc điểm:** Chậm hơn unit tests, kiểm tra luồng end-to-end hoặc các slice cụ thể.

22.2.2.1. `@SpringBootTest`

Annotation `@SpringBootTest` là annotation chính cho integration testing trong Spring Boot. Nó sẽ khởi động toàn bộ Spring ApplicationContext hoặc một phần của nó.

Java

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;

```

```

import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class ProductControllerIntegrationTest {

    @LocalServerPort
    private int port;

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void testGetAllProducts() {
        ResponseEntity<String> response =
restTemplate.getForEntity("http://localhost:" + port + "/products",
String.class);
        assertEquals(HttpStatus.OK, response.getStatusCode());
        assertNotNull(response.getBody());
        // Thêm các assertion khác để kiểm tra nội dung JSON
    }

    @Test
    void testGetProductById() {
        // Giả sử có một sản phẩm với ID 1 trong database test
        ResponseEntity<String> response =
restTemplate.getForEntity("http://localhost:" + port + "/products/1",
String.class);
        assertEquals(HttpStatus.OK, response.getStatusCode());
        assertNotNull(response.getBody());
    }
}

```

- `webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT` : Khởi động một máy chủ web nhúng trên một cổng ngẫu nhiên, tránh xung đột cổng.
- `@LocalServerPort` : Inject cổng ngẫu nhiên được sử dụng.
- `TestRestTemplate` : Một tiện ích để thực hiện các HTTP request đến ứng dụng đang chạy.

22.2.2.2. Slice Testing (Kiểm thử lát cắt)

Slice testing là một dạng của integration testing, nơi bạn chỉ khởi động một phần nhỏ của Spring ApplicationContext, tập trung vào một "lát cắt" cụ thể của ứng dụng (ví dụ: chỉ tầng web, chỉ tầng JPA). Điều này giúp các bài kiểm thử chạy nhanh hơn so với việc khởi động toàn bộ context.

- **@WebMvcTest** : Kiểm thử Controller. Chỉ khởi tạo các bean liên quan đến Spring MVC, không tải các bean Service hay Repository.
- **@DataJpaTest** : Kiểm thử Repository. Chỉ cấu hình các bean liên quan đến JPA (DataSource, EntityManager, Spring Data Repositories). Thường sử dụng cơ sở dữ liệu nhúng (ví dụ: H2) và rollback transaction sau mỗi test.
- **@JdbcTest** : Kiểm thử các thành phần JDBC.
- **@RestClientTest** : Kiểm thử các client REST (ví dụ: `RestTemplate` , `WebClient`).

22.3. Các tiện ích kiểm thử khác

- **@MockBean** : Tạo một mock object cho một bean trong ApplicationContext và inject nó vào các bean khác. Hữu ích khi bạn muốn cô lập một phần của ứng dụng và kiểm soát hành vi của các dependency.
- **@SpyBean** : Tạo một spy object cho một bean hiện có trong ApplicationContext. Spy cho phép bạn gọi các phương thức thực của đối tượng trong khi vẫn có thể verify hoặc stub một số phương thức nhất định.
- **@TestPropertySource** : Ghi đè các thuộc tính cấu hình trong quá trình kiểm thử.
- **@ActiveProfiles** : Kích hoạt các Spring profiles cụ thể cho một bài kiểm thử.
- **@DirtiesContext** : Đánh dấu rằng ApplicationContext đã bị thay đổi và cần được làm sạch (hoặc khởi tạo lại) sau khi test chạy. Nên sử dụng cẩn thận vì nó làm chậm quá trình test.
- **Testcontainers**: Một thư viện Java cung cấp các instance cơ sở dữ liệu, message brokers, hoặc bất kỳ thứ gì khác có thể chạy trong Docker container, cho phép bạn chạy

integration tests với các dịch vụ thực tế mà không cần cài đặt chúng trên máy cục bộ.

22.4. Bản chất của Spring Boot Testing

Bản chất của Spring Boot Testing là cung cấp một môi trường kiểm thử toàn diện và linh hoạt, giúp các nhà phát triển dễ dàng viết và chạy các bài kiểm thử ở nhiều cấp độ khác nhau (unit, integration, slice). Bằng cách tận dụng các annotation chuyên biệt như `@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest` và tích hợp với các framework kiểm thử phổ biến như JUnit và Mockito, Spring Boot giảm thiểu đáng kể công sức cấu hình và boilerplate code cần thiết cho việc kiểm thử. Nó khuyến khích việc kiểm thử tự động, giúp đảm bảo chất lượng phần mềm, phát hiện lỗi sớm và tăng cường sự tự tin khi thay đổi mã nguồn. Mục tiêu cuối cùng là giúp xây dựng các ứng dụng Spring Boot mạnh mẽ, đáng tin cậy và dễ bảo trì.